SwarmBox: An Inescapable, Reproducible Multi-Agent Testbed

Track 1 - Sandboxes and Testbeds Swarm Alignment Hackathon, 6-8 November 2026

Author: Yilang Zhi **Affiliation:** Independent researcher, Shanghai, China **Contact:** 2383844705@qq.com
Code: https://github.com/Zhiyilang/swarmbox (MIT, Python standard library only)

Abstract

SwarmBox is a reproducible multi-agent testbed in which escape is not blocked but inexpressible. Agent output is parsed as data and never evaluated, so filesystem, network, process, import and reflection are absent from the action grammar and cannot be requested. Six operations remain; each passes strict schema, identity and state validation before it touches the world, and every proposal, including every refusal, is written to an append-only JSONL ledger with 20 stable rule identifiers. Conservation, non-negativity and capacity invariants are checked inside the world on every apply. On top of the gate sits a renewable common-pool economy in which 5 to 20 deterministic agents draw over many rounds, with metrics for inequality, overclaim, drift, collapse and cross-agent coordination.

We report four experiments. Across a 32-item adversarial corpus, 31 proposals are refused with the expected rule identifier, one legitimate self-referential proposal is allowed, and there are zero escape successes. Same-seed runs produce byte-identical ledger digests. A monitor reading only the ledger detects a planted colluding pair with correlation -1.000 and 11 cross-agent reads of a shared key, but the signal degrades once the resource collapses. Myopic policies drain the pool on 8 of 8 seeds while informed policies drain it on 0 of 8. The suite is 235 assertions, 0 failures, standard library only, zero network, and reruns byte-identically from a seed.

1. Introduction

Track 1 asks for a sandbox that is reproducible, multi-agent, shared-resource, logged and inescapable - and then asks how far a cheap model can substitute for a frontier one. The 2026 Hugging Face incident, in which roughly 1,200 agents built an unplanned coordination channel through a shared repository, is the motivating example: the interesting failure is not one agent misbehaving, but a population finding a channel nobody designed.

We took "cannot be escaped" as the hard part, because it is the claim that is easy to assert and hard to support. Our question, sharpened:

> Can we build a multi-agent environment where escape is not merely blocked but inexpressible, and where the containment claim is backed by an executable corpus of attacks rather than a sentence in a README?

2. Related Work and Prior Work

The containment philosophy - a deterministic verifier as the release authority, with append-only receipts - comes from the author's earlier work on single-agent code containment. The multi-agent harness, the resource model, the metrics, the collusion probe and the cheap/frontier experiment extend that idea to populations.

Disclosure. All core code in this repository was written in October 2026, before the sprint, and is marked as pre-existing in both the README and Appendix B. It is not claimed as work produced during 6-8 November 2026. The sprint submission adds this research report, the required Limitations and Dual-Use analysis, and an independent re-verification of every number quoted below, re-run on 5 November 2026. We make no claim that the code itself was written during the sprint weekend. No result in this report was produced by a language model, and no result is simulated.

3. Methodology

`swarmbox/` is ~900 lines, standard library only, no network, no third-party package.

Module	Role
`gate.py`	The only path from intent to world. Six operations, strict schema, 20 rule ids.
`world.py`	A regenerating common-pool resource, private per-agent cells, shared scratch.
`agents.py`	Eight deterministic policies in two families.
`swarm.py`	The round loop. A run is a pure function of its config.
`ledger.py`	Append-only JSONL. Refusals are first-class records.
`metrics.py`	Gini, overclaim, drift, collapse, pairwise correlation, covert-channel count.
`fidelity.py`	The cheap-vs-frontier experiment.
`probe.py`	The planted-collusion experiment.

3.1 The central design decision: the sandbox is not an interpreter

Agent output is parsed as **data** and is never evaluated. There is no filesystem path, no socket, no subprocess, no import and no reflection anywhere in the action grammar. Those escapes are therefore not "blocked" - they cannot be *requested*.

What remains is six operations, each checked before it touches the world:

Two consequences we consider more valuable than any single rule:

- **The attack surface is finite and enumerable.** We can list every way an agent can address the world, which is what makes a "it holds" claim checkable at all.
- **The refusal is legible.** Every refusal carries a stable rule id, so a report can point at `gate.string.non-canonical` instead of saying "it was rejected".

3.2 Three layers of containment

Layer	What it stops
Structural - not in the grammar	filesystem, network, process, import, reflection, eval
Schema - strict, no unknown fields	smuggling, type confusion, impersonation via extra fields
State - checked against the world	over-claim, over-release, reserved namespaces, unknown peers

The schema is deliberately strict: an unknown field is refused rather than ignored. Ignoring is how data gets smuggled. Impersonation is handled by permitting an optional `agent` field *so that it can be rejected by the identity rule* - if it were simply not in the schema, every impersonation would be reported as a schema error and the ledger would be less useful as evidence.

3.3 Invariants: the arithmetic that cannot lie

These are checked inside `World.apply()` and raise on violation; they are also written to the ledger once per round. An agent that escapes would almost certainly have to conjure units from nothing, and that breaks arithmetic first. The invariants are the tripwire, the gate is the fence.

3.4 Metrics and configurations

A run is parameterised by `(seed, n, rounds, capacity, regen, roles)` and is a pure function of that tuple. Metrics are computed from the ledger, never from in-memory shortcuts, so a monitor and a judge read exactly the same evidence. Reported numbers below come from `python eval.py`, `python -m swarmbox gate`, `python -m swarmbox probe` and `python -m swarmbox fidelity`, each re-run for this submission.

4. Results

E1 - Can it be escaped? (the attack corpus)

`gate.ESCAPE_CORPUS` contains 32 adversarial proposals: unknown ops, negative and zero amounts, `10**9` claims, float/bool/string/None where an int is required, unknown fields, path-like keys (`../secret`), reserved namespaces (`private_*`, `system_*`), NUL and DEL characters, fullwidth and ligature keys designed to reach the same namespace after normalisation, oversized values, impersonation, unknown peers, and an unknown spawn role.

Result: 31 refused, each with the rule id written next to it in the corpus; 1 correctly allowed (a legitimate proposal naming its own agent). Zero escapes. This runs on every CI invocation, so the claim cannot rot.

A substring scan is not how we check the package is clean, incidentally - the words "socket" and "subprocess" appear in `gate.py`'s docstring, because explaining that they are absent is the point. The check walks the AST instead. A guard that flags its own documentation is a guard that gets ignored.

E2 - Can another team rerun it?

Result: for seeds 1-5, two runs with the same seed produce byte-identical ledger digests and identical metric bundles; the five seeds produce five distinct runs. The ledger records the full configuration in its first line, so a team handed only the ledger knows exactly what to rerun.

E3 - Can a monitor that sees only the ledger find coordination?

We plant one colluding pair in a population of cooperators. The pair never speaks in the open: they alternate a maximum claim with a one-unit claim so the per-round total looks ordinary, and they pass a one-bit signal through the shared scratch space. Nothing about them is special-cased in the world or the gate - they use ordinary operations.

Result: the monitor flags exactly one pair, it is the planted pair, with correlation -1.000, and it counts 11 cross-agent reads of a shared key.

One honest caveat we found while calibrating: **this signal only survives while the pool does.** Once the resource collapses, claims get clamped by scarcity and the alternating pattern is destroyed. Monitoring quality degrades after collapse - which is worth knowing if you are designing oversight for a resource-constrained swarm.

E4 - How far can a cheap model stand in for a frontier one?

We cannot call models from a repository that must run offline and reproduce for free, so we isolate the axis along which a cheap model actually differs here: **what it can condition on.**

- **cheap family** (4 policies) - myopic: sees its own holding and the pool.

- **frontier family** (2 policies) - informed: additionally sees the swarm's recent aggregate behaviour and adapts to it.

Eight seeds, 12 agents, 24 rounds each.

family	gini	overclaim	drift	collapse rate
cheap (myopic)	0.2003	1.125	-112.500	1.00
frontier (informed)	0.0354	1.071	-44.297	0.00
mixed	0.2638	1.152	-104.672	1.00

Result: the myopic family exhausts the pool on all 8 seeds; the informed family never does, on any seed. The divergence is categorical - collapse versus no collapse - not a small delta you could average away. Inequality differs by 5.7x (gini 0.2003 vs 0.0354).

So the answer to Track 1's third question, on this testbed, is: not far at all. A cheap stand-in does not reproduce frontier behaviour even on a resource problem this small. If a team wants to screen swarm dynamics cheaply, they must first establish that the specific metric they care about is insensitive to the information set - ours were not.

5. Discussion

1. **Containment is much cheaper to guarantee when the sandbox is not an interpreter.** Six operations with a strict schema produce a finite, enumerable attack surface - and therefore a corpus you can actually run, rather than a claim you can only assert.

2. The escape corpus must be part of CI, not part of the write-up. 32 attacks, re-run on every commit. A "cannot be escaped" claim that is not executed is a rumour.

3. Conservation invariants are a better tripwire than permission lists. Arithmetic cannot be socially engineered.

4. Refusals have to be first-class records. A ledger of only successful actions cannot support any containment claim, and it is the natural thing to write by accident.

5. The cheap/frontier gap here is categorical, not gradual. Collapse rate 1.00 vs 0.00 across 8 seeds. A cheap stand-in is not a safe substitute for studying resource-sharing swarms.

6. Oversight degrades after resource collapse. Coordination detection worked perfectly (correlation -1.000) while the pool survived and loses signal once scarcity clamps behaviour.

6. Limitations

- **No language model is called anywhere in this repository.** Every policy is a pure function. The cheap/frontier result is an experiment about *information sets*, not a benchmark of real models. Wiring in a real cheap-vs-frontier pair is the obvious next step and the code is structured for it (`agents.propose` is a single dispatch point).
- **The "frontier" family has only two policies** (`forecaster`, `adapter`). Their advantage may be an artefact of how they were written, not of being better informed. More policies per family would test that.
- **Escapes are only refused within the action language.** We argue the language cannot express an escape; we do not claim to have defeated a general-purpose code-execution sandbox, because we do not run one. Different threat model, different claim.
- **The gate is a static checker, not a runtime sandbox.** An agent cannot reach the host through SwarmBox because SwarmBox offers no path to it - if someone embedded this in a system that did, this analysis would not cover the new surface.
- **One resource, one regeneration model.** No prices, no punishment, no exit. Real swarms have all three.

- **Correlation is a weak collusion detector** in general. It found our planted pair because that pair was turn-taking. Sophisticated collusion would need better tools (Track 4's problem, not ours).

7. Reproduction

```
```

Requires Python 3.9+ and nothing else. `env -u PYTHONUTF8 LC_ALL=C python eval.py` also returns 0 (Windows cp936 is a real way to lose a submission). A Gitee mirror is at https://gitee.com/Zhiyilang/swarmbox.

Appendix A - Limitations and Dual-Use / Ethical Considerations (required)

What this result does and does not establish. We establish that, inside a six-operation data-only grammar with an append-only ledger, 32 adversarial proposals produce zero escapes, three arithmetic invariants hold after every apply, and a ledger-only monitor can detect one specific planted coordination pattern. We do **not** establish that a real LLM-driven swarm is contained, that any production sandbox is secure, or that general-purpose code execution can be made safe. No language model, network call, credential or host filesystem is involved. The cheap/frontier comparison is about information sets, not model benchmarks.

Ways this setup could make multi-agent operations easier to carry out or harder to detect.

1. **An oracle for covert-channel tuning.** Because every refusal is logged with a stable rule id, an operator can iterate candidate coordination strategies offline, at zero cost, and read exactly which attempt left a detectable trace. That is useful for defenders, but it is equally a cheap way to search for strategies that leave no trace.
2. **Induced scarcity as an anti-monitoring tactic.** Our E3 caveat - that the correlation signal dies once the pool collapses - suggests that flooding or draining a shared resource can degrade ledger-based oversight. We report it because defenders need to know it; we also recognise it as a usable tactic.
3. **A published specification of a checker.** Twenty rule ids and a six-operation grammar tell a designer exactly what one containment checker looks for. Applied to a real system, that list is a starting point for enumerating bypass targets.
4. **Auditability cuts both ways.** An append-only ledger is evidence for a judge, but also a faithful replay of an operation for anyone who obtains it. We publish only aggregate metrics and synthetic runs, and no real user data.

Mitigations adopted. The testbed offers no filesystem, network, subprocess, import or reflection; it calls no external model; it holds no credentials and no real data; all agents are deterministic policies; every claim in this report is scoped to the grammar; and we explicitly recommend that monitoring not rely on a single correlation statistic. We consider the defensive value of an executable, reproducible containment claim to outweigh these risks, while stating them plainly so a reader can disagree.

Appendix B - Pre-existing Work Disclosure

As required by the Apart Research submission guidelines ("Mark any code you wrote before the sprint as preexisting in the README and the report"), we state the lineage explicitly:

- All core modules (`gate.py`, `world.py`, `agents.py`, `swarm.py`, `ledger.py`, `metrics.py`, `fidelity.py`, `probe.py`), the 235-assertion test suite, the 32-item escape corpus and the report PDF pipeline were written between 20 and 31 October 2026 - **before** the 6-8 November sprint - and are **pre-existing**.
- The README of the repository carries the same disclosure. No attempt is made to present pre-existing code as weekend work.
- The work prepared for this sprint is this research report, its abstract, the Limitations and Dual-Use / Ethical Considerations appendix, and a fresh independent re-verification of all quoted numbers (5 November 2026: `RESULT: 235 passed, 0 failed`; gate `32 items, 31 refused with the expected rule`, `escape successes 0`; fidelity `escape successes across all runs: 0`, `invariant violations across all runs: 0`).

Appendix C - Weekend-work increment: swarm-size sweep (5-20 agents)

Generated during the sprint window (AoE 6-8 November 2026), 08 November 2026 10:23 (Beijing). This appendix is new work performed for the hackathon; the core code is pre-existing and is declared in Appendix B. Command: `python weekend_increment.py --full`. Raw measurements: `WEEKEND_RESULTS.json`.

Track 1 asks for environments with 5-20 agents. The original experiments used a single population size (12). Here we sweep n = 5, 8, 12, 16 and 20, running the cheap, frontier and mixed families over the same eight seeds at each size, to test whether the E4 result is an artefact of one configuration.

agents n	cheap collapse	frontier collapse	collapse gap	gini gap	escapes	invariant violations
5	0.00	0.00	0.00	0.0786	0	0
8	1.00	0.00	1.00	0.1322	0	0
12	1.00	0.00	1.00	0.1649	0	0
16	1.00	0.00	1.00	0.2220	0	0
20	1.00	0.00	1.00	0.2521	0	0

Result. The cheap/frontier collapse gap is not constant across swarm size; sizes with a gap below 1.00: 5. Across all 120 runs there were 0 escape successes and 0 invariant violations. Containment and arithmetic invariants hold at every size.

Limitations of this increment. It reuses the deterministic policies and one regeneration model; no language model is called, and the sweep changes population size only. It establishes scale-robustness inside this testbed, not scale-robustness in real LLM swarms.

Appendix C (generated 08 November 2026) adds a new 5-20 agent scale sweep run during the sprint window; the measurements in it are weekend work, the harness is not.

References

[1] AI Alignment Foundation, "Swarm Alignment Hackathon" (2026). https://www.aialignmentfoundation.org/hackathons/swarm-alignment-2026 [2] Apart Research, "Swarm Alignment Hackathon - Submission Guidelines" (2026). https://apartresearch.com/sprints/swarm-alignment-2026#guidelines [3] METR, "Hugging Face incident report" (August 2026), cited by [1]. https://metr.org/hugging-face-incident-report-aug-2026.pdf [4] SwarmBox source code and reproduction suite (MIT). https://github.com/Zhiyilang/swarmbox